

10384 Quantum Information – exercises, week 9

Pen-and-paper exercises

1. Show that $\hat{H}^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{x \cdot y} |y\rangle$ with $x \cdot y = \sum_{i=0}^{n-1} x_i y_i \pmod{2}$.
You can start with the simple cases of $n = 1, 2$ and convince yourself that it generalises to arbitrary n .
2. Find the bra-ket and matrix form of the unitary operations implementing all four possible single-bit ($\{0, 1\} \rightarrow \{0, 1\}$) functions using the standard model with 2 qubits (1 input, 1 output), $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$.
3. Construct quantum circuits that can act as oracles (i.e., implement $\hat{U}_f$) for all four 1 bit functions.
4. Construct 4-qubit quantum circuits that implement $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$ for a few different constant and balanced functions $f: \{0, 1\}^3 \rightarrow \{0, 1\}$. These will serve as oracles in the computational exercises.
5. Prove unitarity of the general $n \rightarrow m$ ($\{0, 1\}^n \rightarrow \{0, 1\}^m$) quantum processor $\hat{U}_f|a\rangle \otimes |b\rangle = |a\rangle \otimes |b \oplus f(a)\rangle$.
Hint: Start by writing a compact expression for the bra-ket form of $\hat{U}_f$.

Computational exercises

If you haven't already done so, learn about QuTiP from the `QI - introducing QuTiP.ipynb` notebook [here](#) and/or from the official documentation for [QuTiP itself](#) and for the [qutip-qip subpackage](#) for quantum information processing.

6. Using QuTiP, implement the Deutsch-Jozsa algorithm on the oracles you constructed in exercise 4.
You can use this template:

```
import qutip as q
from qutip.qip.circuit import QubitCircuit

oracle1 = QubitCircuit(4)
oracle1.add_gate( [...] )
[...]
```

```
dj = QubitCircuit(4, reverse_states=False, num_cbits=3)
dj.add_gate( [...] )
[...]
```

```
dj.add_circuit(oracle1)
[...]
```

```
for i in range(3):
    dj.add_measurement('meas', targets=i, classical_store=i)
```

```
dj
```

To check the classical measurement outcomes of a circuit, run `result = dj.run_statistics(q.ket('0000'))`. This initialises the circuit with the state $|0000\rangle$ and stores the measurement outcomes in `result.cbits` and their corresponding measurement probabilities in `result.proBABILITIES`.

Do the constant function oracles always return 000? Do the balanced function oracles never return 000?

7. The Bernstein-Vazirani algorithm (see Aaronson 18.1) is closely related to the Deutsch-Jozsa algorithm.

In fact, the quantum circuit is completely identical, whereas the problem, the oracle, and the interpretation of the measurement results are different:

Assuming you have a black-box oracle implementing a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ that is promised to be given by $f(x) = x \cdot s \pmod{2}$ for some unknown binary string s of length n . The objective is then to query the oracle to determine the string s . Classically, you must query the oracle n times, but with the Bernstein-Vazirani quantum algorithm only a single query is needed.

Re-using your circuit from exercise 7, demonstrate the Bernstein-Vazirani algorithm using an n of your own choice and a number of choices of s . You will need to design a new oracle for f . This can be done using only CNOT gates: Note that by the definition of f , the value of the single-qubit output register flips for every digit that is simultaneously 1 in both x and s .